

Documentação Técnica

Solução: EcoPulse

Domain Driven Design using Java

Equipe: EcoPulse

Projeto: Challenge EcoPulse

Data: 13/09/2026

Versão: 1.0

Integrantes

Pedro Henrique Carvalho do Nascimento — RM 570492

Gabriel Pereira de Oliveira — RM 572262

Adalto Massashiro Nagabe — RM 572298

Kauã Barros Ferreira — RM 571801

Murillo Siviero Lopes — RM 57272

Sumário

1	1. Objetivo e escopo	1
2	2. Funcionalidades implementadas	1
	2.1 2.1 Gerenciamento de usuários	1
	2.2 2.2 Gerenciamento de avatares	1
	2.3 2.3 Gerenciamento de missões	1
	2.4 2.4 Persistência e inicialização	1
	2.5 2.5 Tratamento de erros	2
3	3. Protótipo de telas	3
	3.1 3.1 Tela inicial	3
	3.2 3.2 Tela Sobre o EcoPulse	4
	3.3 3.3 Tela Sobre o Souly	5
	3.4 3.4 Tela Souly como guia interativo	6
	3.5 3.5 Tela de integrantes	7
	3.6 3.6 Tela de perguntas frequentes	8
	3.7 3.7 Tela de contato	9
4	4. Arquitetura da solução	9
5	5. Diagrama do banco de dados	10
6	6. Diagrama de classes	11
7	7. Ferramentas e versões necessárias	12
8	8. Procedimentos para importar e executar	12
	8.1 8.1 Importação no IntelliJ IDEA	12
	8.2 8.2 Configuração Oracle FIAP	12
	8.3 8.3 Compilação	13
	8.4 8.4 Execução do menu interativo	13
	8.5 8.5 Execução dos testes automatizados	13
	8.6 8.6 Execução do teste manual de CRUD	13
9	9. Limites e observações	14
10	10. Referências	14

1 1. Objetivo e escopo

O Challenge EcoPulse é uma aplicação Java de console criada para apoiar uma experiência de incentivo a hábitos sustentáveis. O sistema organiza usuários, seus avatares e missões ambientais em um domínio orientado a objetos, com persistência em Oracle Database por meio de JDBC.

O propósito principal é permitir que um usuário seja cadastrado, tenha um avatar associado e participe de uma dinâmica de evolução baseada em experiência. O sistema também oferece o cadastro e a administração de missões, incluindo ativação, desativação, atualização e controle de pontuação.

O escopo desta entrega contempla o fluxo de console para usuários, avatares e missões; as regras de negócio dessas entidades; a conexão Oracle; a criação inicial do schema; os DAOs com operações CRUD; as exceções específicas; e os testes automatizados das regras de domínio. O projeto não implementa autenticação, autorização, interface web, aplicativo mobile, processamento assíncrono ou integração com serviços externos.

A solução está estruturada segundo uma separação de responsabilidades. As entidades concentram o comportamento do domínio, os services orquestram regras e casos de uso, os DAOs executam a persistência JDBC e a interface de console coleta entradas e apresenta resultados.

2 2. Funcionalidades implementadas

2.1 2.1 Gerenciamento de usuários

O módulo de usuários permite criar, listar, pesquisar por nome, consultar por identificador, atualizar perfil, exibir resumo e excluir registros. O UserService normaliza nome e e-mail, valida o formato mínimo do cadastro e delega as operações de persistência ao UserDao.

A entidade User possui regras para produzir o nome de exibição, verificar se o perfil está completo, validar o e-mail e atualizar os dados editáveis do perfil.

2.2 2.2 Gerenciamento de avatares

O módulo de avatares permite criar um avatar vinculado a um usuário existente, listar avatares, adicionar experiência, registrar interação e excluir registros. A criação valida a existência do usuário proprietário antes de executar o INSERT, evitando violação de chave estrangeira.

A entidade Avatar calcula evolução, experiência necessária para o próximo nível, nome de exibição e atualização da última interação. A experiência positiva pode elevar automaticamente o nível do avatar.

2.3 2.3 Gerenciamento de missões

O módulo de missões permite criar, listar todas, listar somente as ativas, atualizar detalhes, ativar ou desativar e excluir missões. A entidade Mission controla disponibilidade, pontuação, título de exibição, ativação, desativação e atualização dos dados editáveis.

2.4 2.4 Persistência e inicialização

A ConnectionFactory centraliza a conexão Oracle, carrega explicitamente o driver JDBC e cria as tabelas ausentes na primeira inicialização. As tabelas utilizam o prefixo obrigatório T_CHLNG_.

Os DAOs UserDao, AvatarDao e MissionDao são a fonte da lógica JDBC dos três agregados principais. Os repositories específicos permanecem como fachadas de compatibilidade e o GenericRepository está disponível para futuras entidades simples.

2.5 2.5 Tratamento de erros

A pasta exception contém exceções específicas para validação de domínio, recurso inexistente e falhas de persistência. A interface interativa captura essas falhas e apresenta mensagens orientadas ao usuário, evitando o encerramento abrupto em erros previsíveis.

Ao selecionar a opção 0 no menu principal, o sistema remove as linhas das tabelas na ordem correta das dependências, mas preserva o schema e as tabelas para a próxima execução.

3 3. Protótipo de telas

O arquivo `ecopulse-front-end-2semestre.zip` fornecido pela equipe contém um protótipo navegável em React e Vite, inspirado nas telas do Figma. As telas abaixo representam a experiência visual planejada para a solução. Elas são apresentadas como protótipo de interface e não substituem o menu de console nem significam que o backend Java já esteja conectado a todas as interações web.

Cada captura foi obtida a partir da execução local do protótipo. O cabeçalho é compartilhado entre as rotas e permite navegar para as páginas principais. Os botões e links descritos abaixo orientam o fluxo esperado do usuário.

3.1 3.1 Tela inicial



图 1 Tela inicial do protótipo — rota /.

Na tela inicial, o usuário conhece a proposta do EcoPulse, visualiza o Souly e lê a mensagem de propósito da solução. O botão **Conheça o Souly** leva à apresentação do guia. O botão **Explore as soluções** desloca a página até os cards de soluções. Os links do cabeçalho levam às telas institucionais, de FAQ, equipe e contato.

3.2 3.2 Tela Sobre o EcoPulse

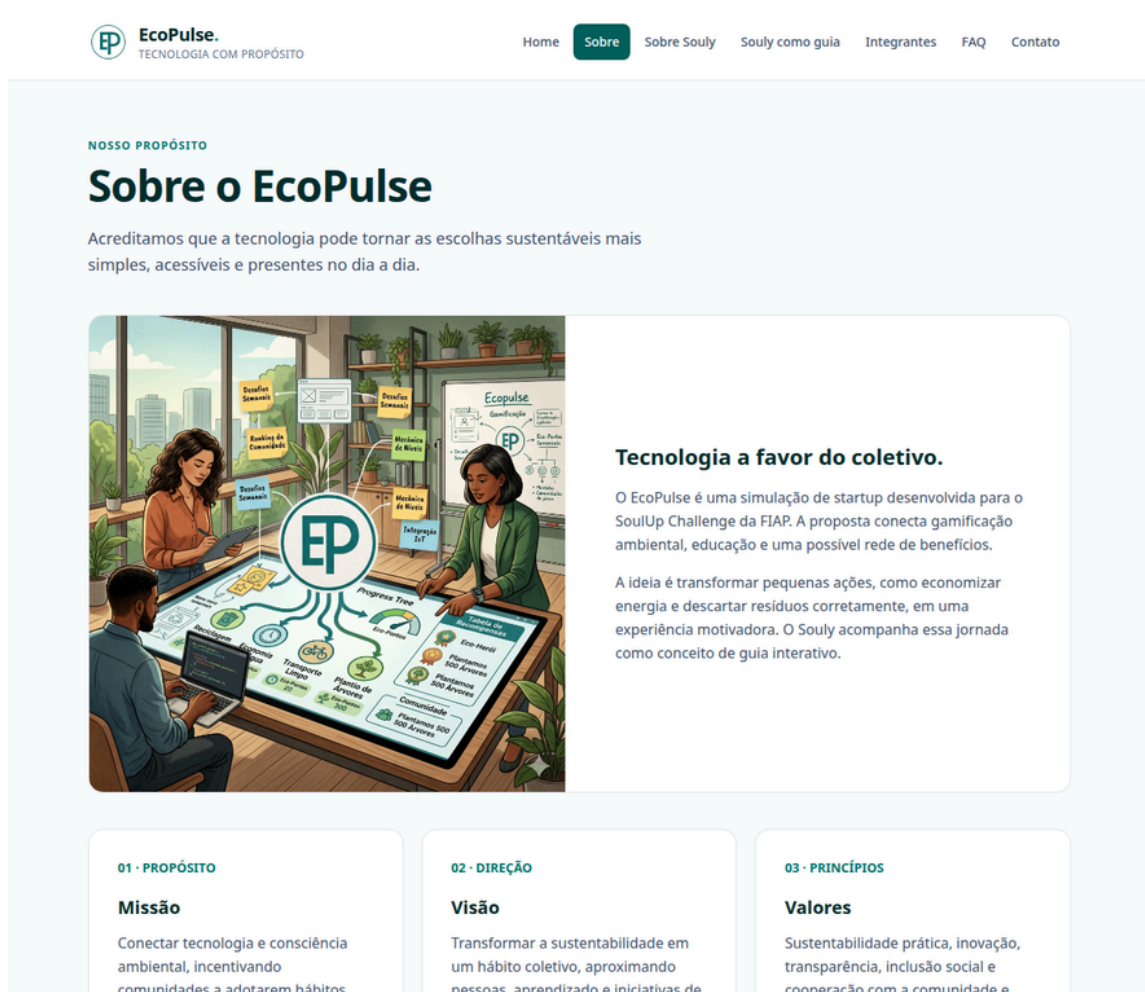


图 2 Tela de apresentação do propósito, missão, visão e valores — rota /sobre.

Nesta tela, o usuário entende o problema que a solução pretende abordar. A seção apresenta propósito, missão, visão e valores. O botão **Conheça a equipe** conduz à tela de integrantes, permitindo relacionar a solução às pessoas responsáveis pelo projeto.

3.3 3.3 Tela Sobre o Souly

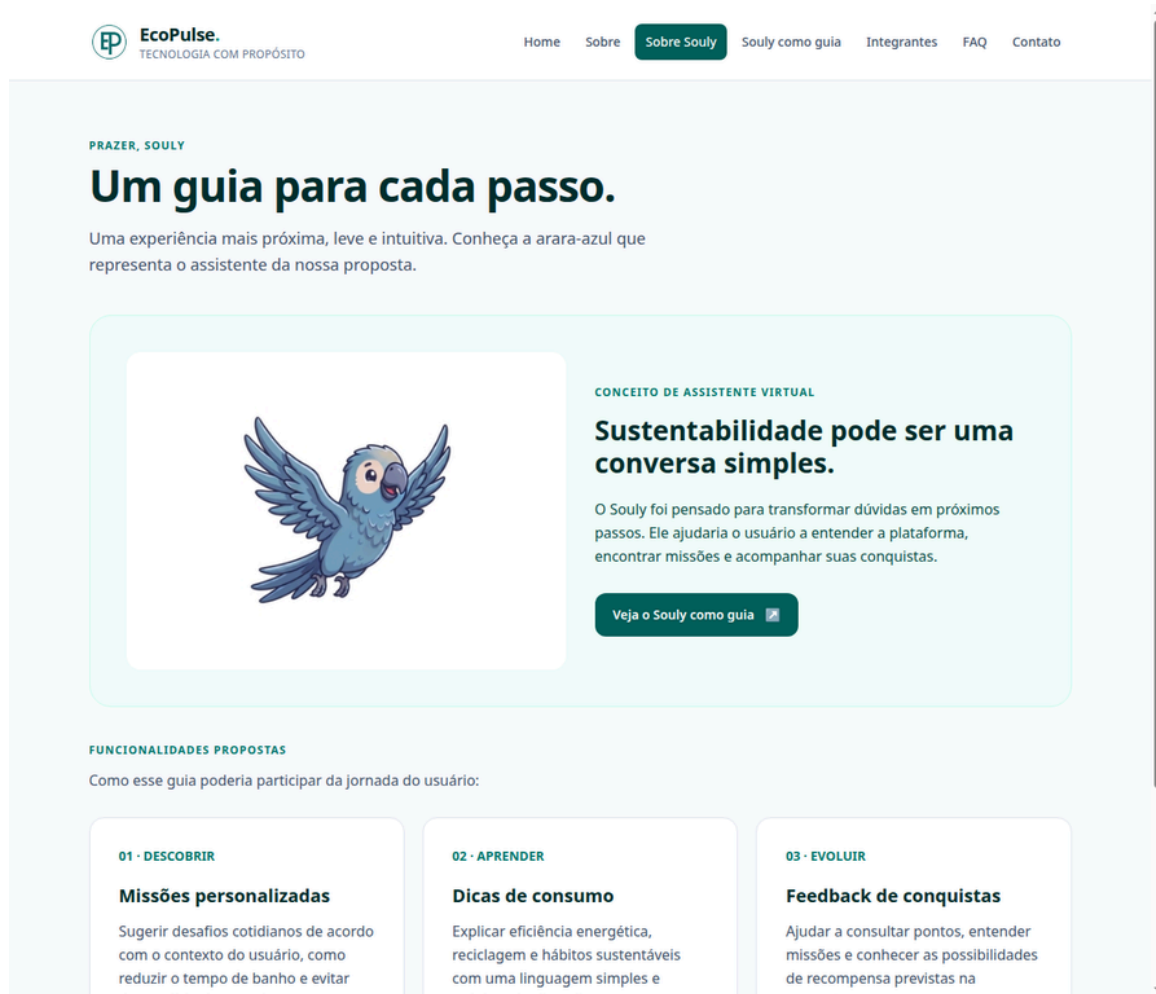


图 3 Tela conceitual do assistente Souly — rota /sobre-souly.

O usuário visualiza o Souly e conhece as funcionalidades propostas para o guia interativo. Os cards apresentam três momentos da jornada: descobrir missões, aprender com dicas de consumo e evoluir acompanhando conquistas. O botão **Veja o Souly como guia** direciona para as demonstrações em vídeo.

3.4 3.4 Tela Souly como guia interativo

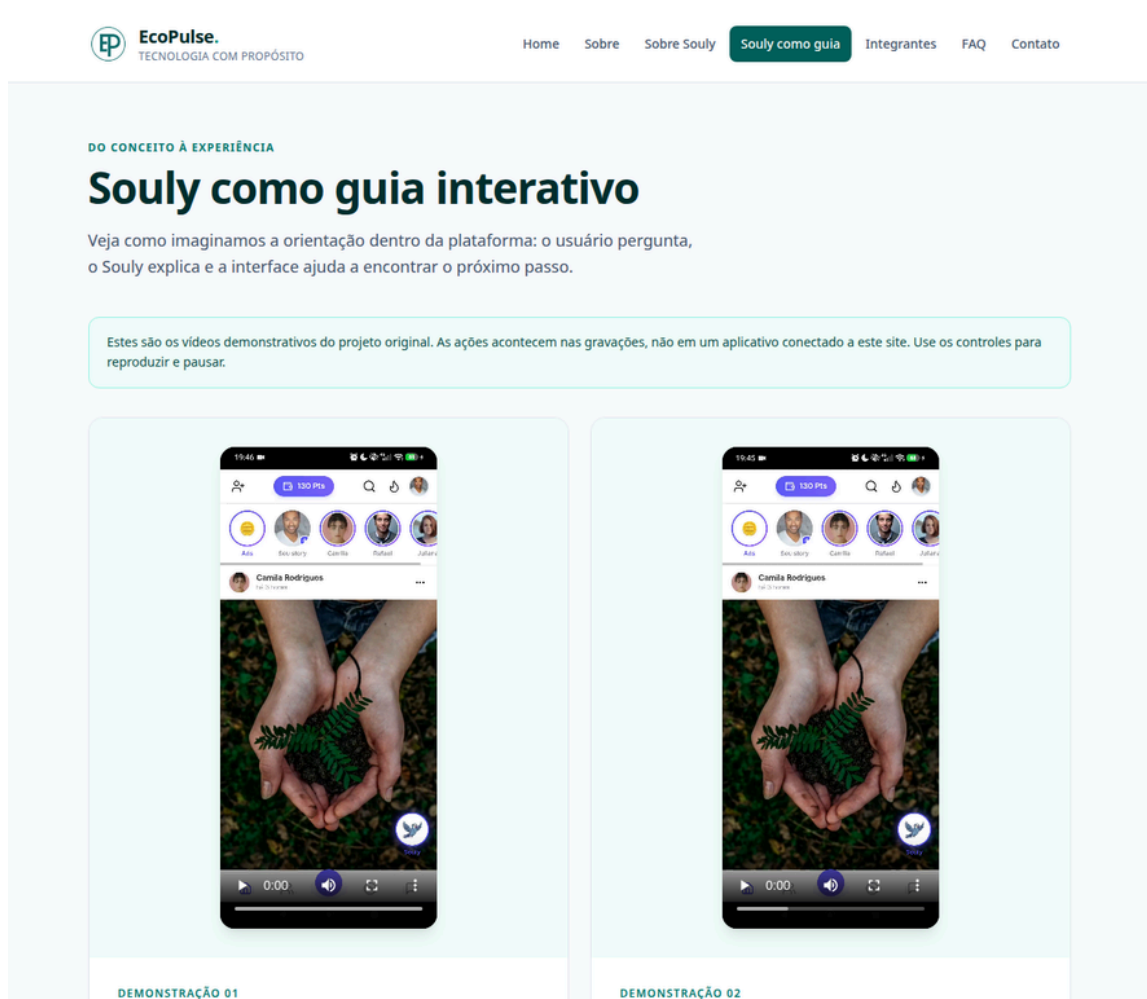


图 4 Tela com demonstrações em vídeo do Souly — rota /souly-como-guia.

O usuário pode reproduzir, pausar e controlar os vídeos demonstrativos. A primeira demonstração apresenta orientação de navegação e a segunda apresenta uma consulta conceitual de recompensas. Os vídeos ilustram o comportamento desejado, mas não representam uma integração real com o banco ou com inteligência artificial nesta versão.

3.5 3.5 Tela de integrantes

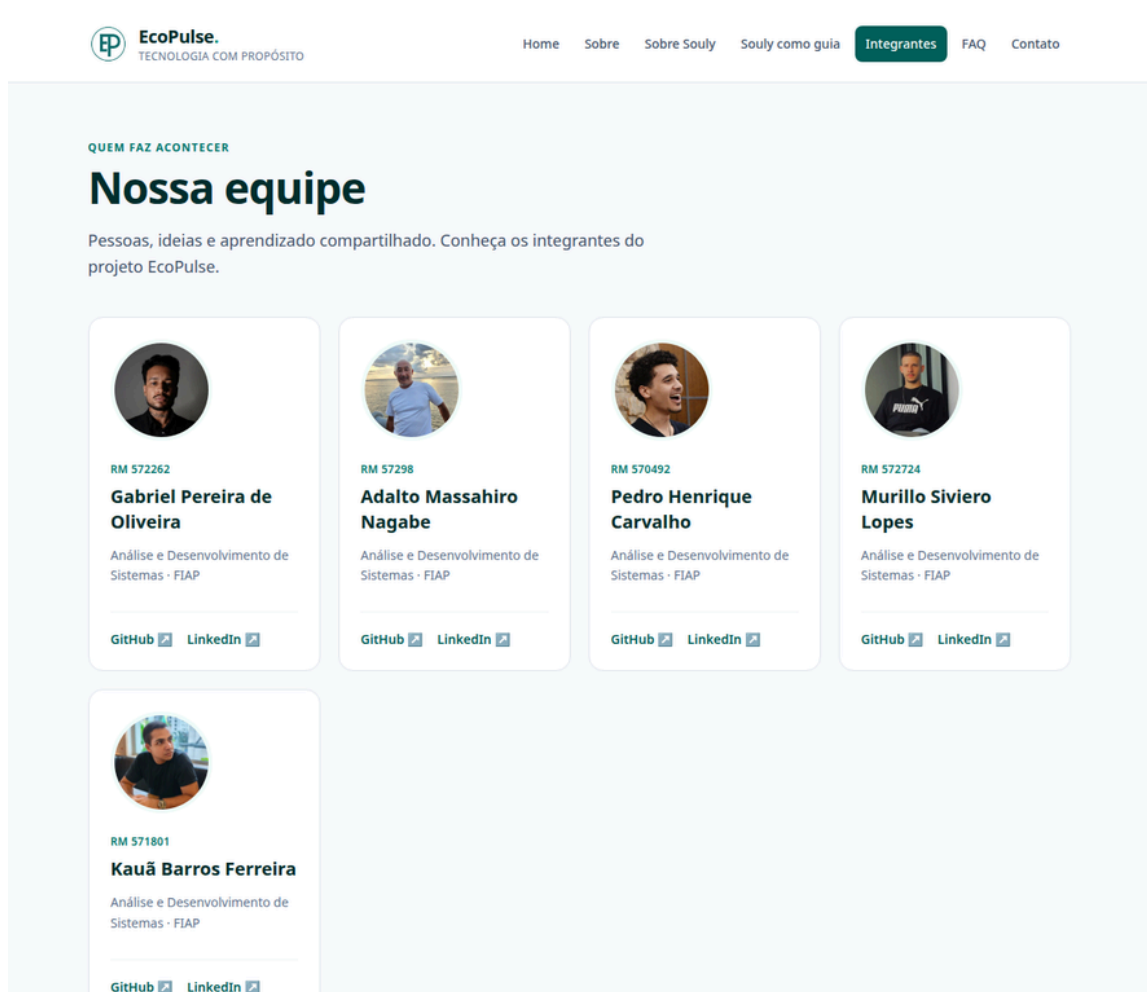


图 5 Tela de apresentação da equipe — rota /integrantes.

Nesta tela, o usuário consulta os integrantes do projeto, seus RMs e seus perfis profissionais. Os links **GitHub** e **LinkedIn** abrem as páginas externas correspondentes. A tela reforça a autoria coletiva da solução e complementa as informações apresentadas na capa desta documentação.

3.6 3.6 Tela de perguntas frequentes

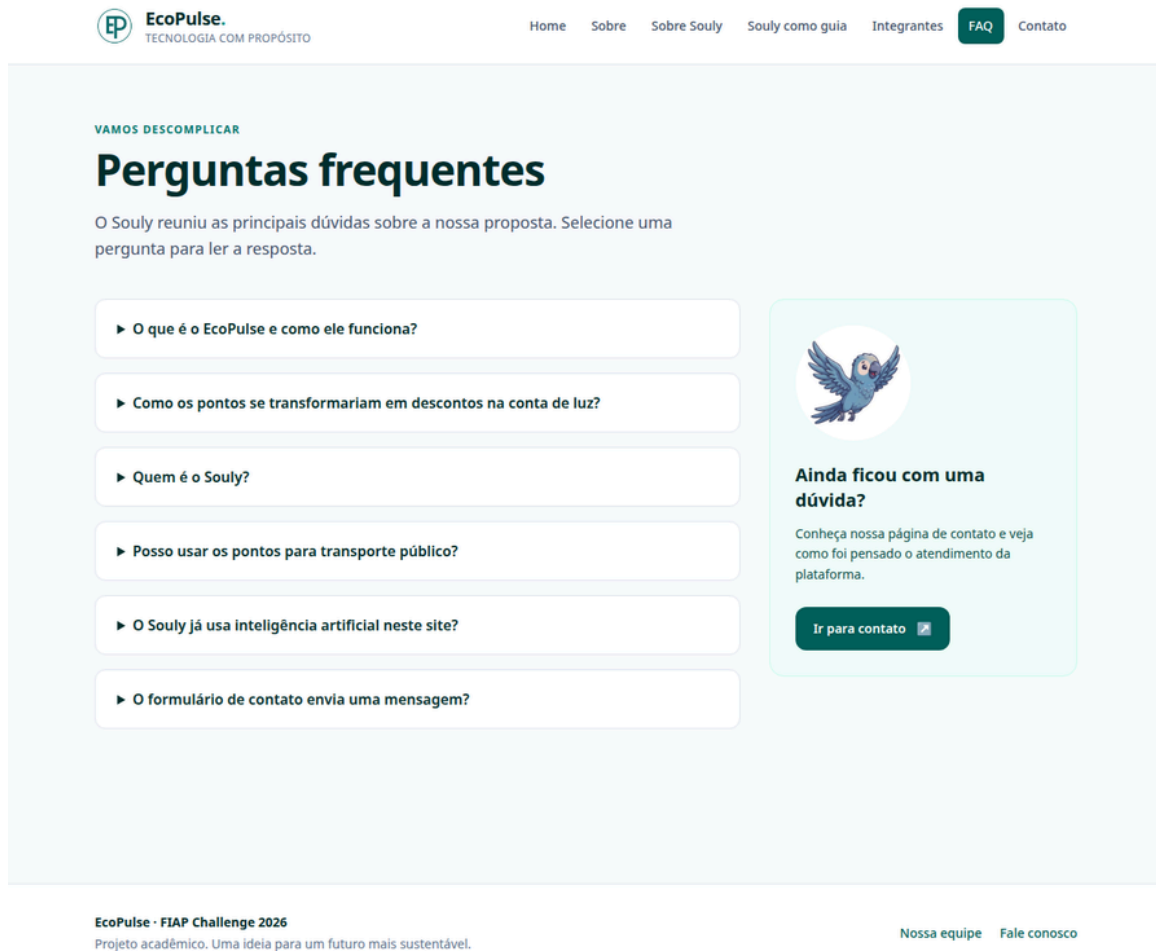


图 6 Tela de perguntas frequentes com respostas expansíveis — rota /faq.

O usuário seleciona uma pergunta para abrir ou fechar sua resposta. O componente usa o padrão de acordeão para concentrar informações sobre pontos, benefícios, Souly, inteligência artificial e limitações do protótipo. O botão **Ir para contato** conduz à tela de formulário.

3.7 3.7 Tela de contato

EcoPulse.
TECNOLOGIA COM PROPÓSITO

Home Sobre Sobre Souly Souly como guia Integrantes FAQ Contato

ESPAÇO PARA CONVERSAR

Fale com a gente

Dúvidas, sugestões ou ideias para o projeto? Veja como foi pensada a experiência de contato do EcoPulse.

Uma boa ideia começa com diálogo.

O formulário ao lado é uma demonstração acadêmica. Preencha os campos para testar a validação básica, sem enviar informações a nenhum serviço.

E-mail ilustrativo do projeto original
suporte@ecopulse.com

Localização do projeto
FIAP - São Paulo, Brasil

Horário de atendimento proposto
Segunda a sexta, das 9h às 18h

Os dados acima fazem parte da simulação. Não representam um canal de suporte ativo.

Teste o formulário

Todos os campos são obrigatórios. Use dados fictícios; nada será enviado ou armazenado.

Nome completo
Ex.: Alex Silva

E-mail
Ex.: alex@example.com

Telefone
Ex.: 11999999999

DDD + telefone: 10 ou 11 números, sem espaços.

Mensagem
Escreva uma mensagem fictícia para testar...

Testar preenchimento

Fig 7 Tela de contato e validação demonstrativa — rota /contato.

O usuário preenche nome, e-mail, telefone e mensagem usando dados fictícios. O botão **Testar preenchimento** executa a validação básica do formulário. O protótipo informa que os dados não são enviados nem armazenados, portanto esta tela representa uma experiência planejada e não um canal de suporte ativo.

4 4. Arquitetura da solução

A arquitetura usa camadas com responsabilidades definidas.

Camada	Responsabilidade
Domínio	Entidades User, Avatar, Mission e demais classes de negócio.
Service	Validação, normalização, orquestração e aplicação das regras de uso.
DAO	Execução de Create, Read, Update e Delete com JDBC Oracle.
Repository	Fachadas específicas e suporte a repositório genérico.

Controller	Ponto de entrada das operações utilizadas pela interface.
Interface	Menus, leitura de entradas e apresentação de mensagens.
Configuração	Conexão, criação do schema e limpeza dos registros.
Testes	Testes JUnit das regras de domínio e teste manual do CRUD.

5 5. Diagrama do banco de dados

O banco é composto por tabelas de usuários, atividades, avatares, missões, recompensas, desafios e associações. As chaves primárias são identificadores numéricos gerados pelo Oracle. As chaves estrangeiras preservam as relações entre usuário, avatar, missão e recompensa.

A tabela T_CHLNG_USERS é a principal referência dos registros de atividades, avatares, sequências, missões do usuário e recompensas do usuário. T_CHLNG_MISSIONS pode referenciar uma recompensa. As tabelas de associação registram participações e vínculos entre os elementos do domínio.

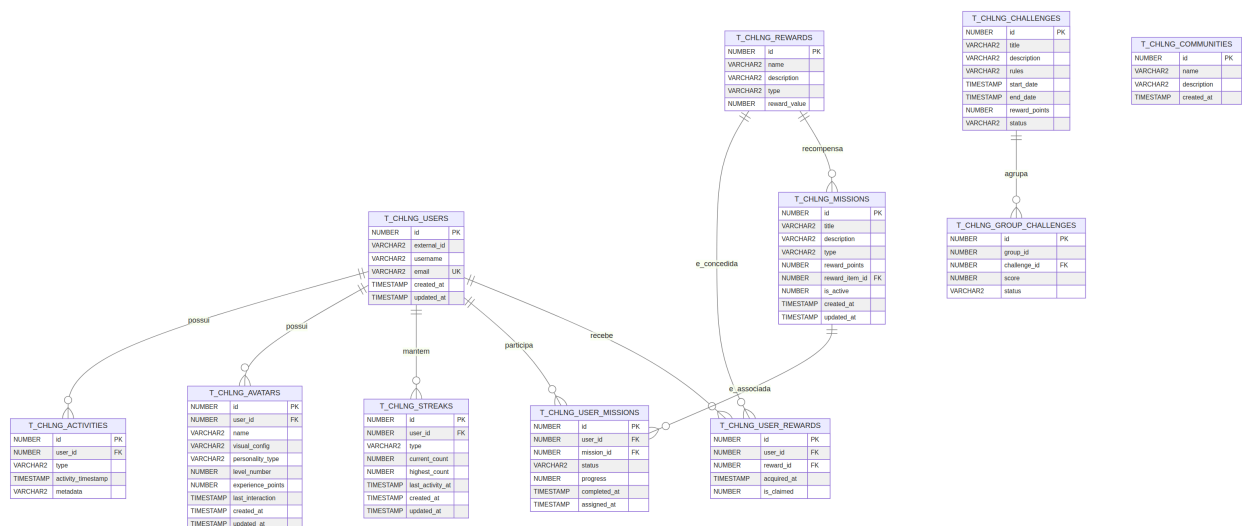


图 8 Modelo entidade-relacionamento do schema Oracle.

6 6. Diagrama de classes

O diagrama apresenta as entidades com seus atributos e regras de negócio, os services que as orquestram, os DAOs responsáveis pela persistência e a ConnectionFactory utilizada pelos DAOs. A dependência entre AvatarService e UserDao representa a validação do proprietário antes do cadastro de um avatar.

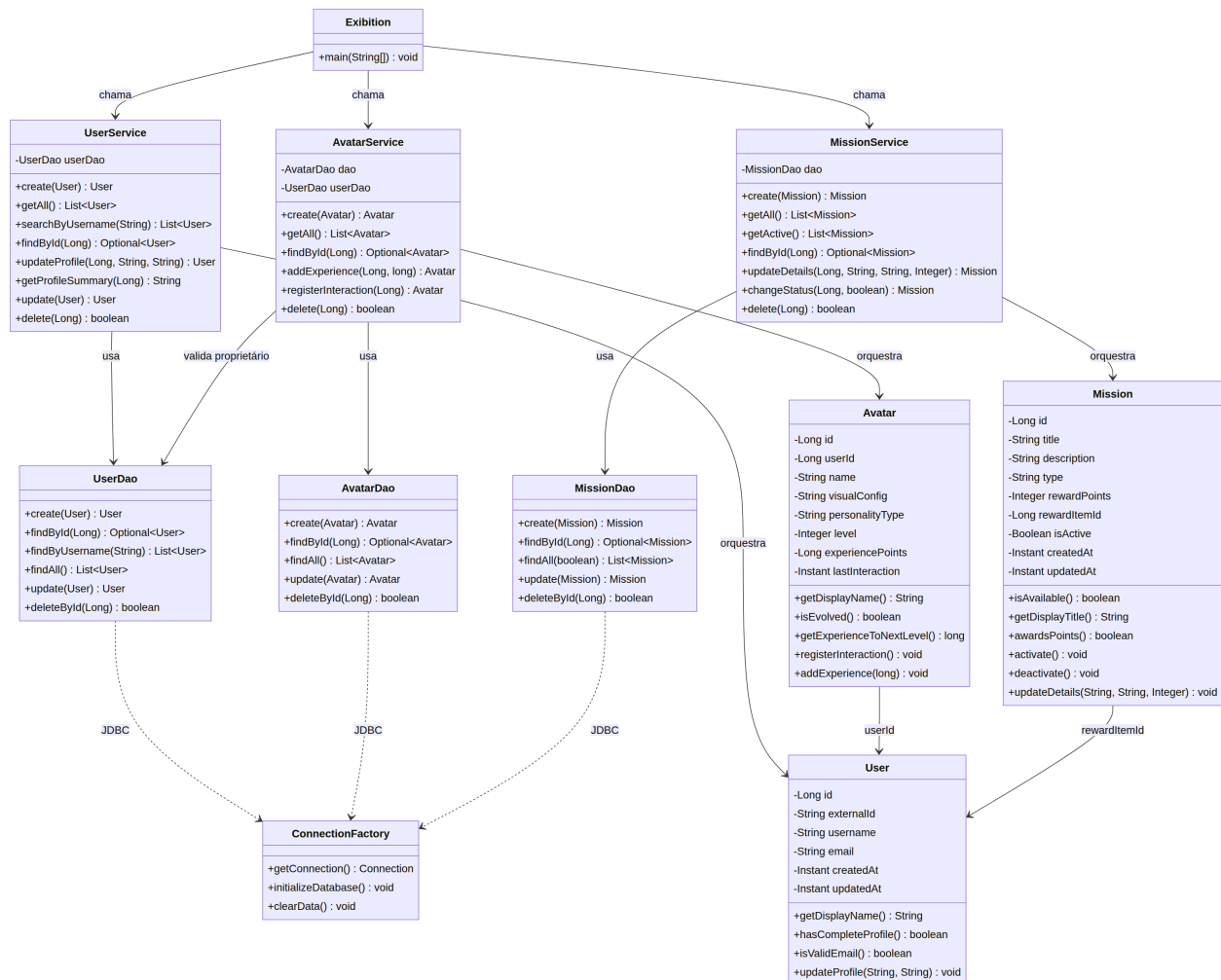


Fig 9 Diagrama de classes da estrutura atual da aplicação.

7 7. Ferramentas e versões necessárias

A execução recomendada utiliza IntelliJ IDEA por sua integração com projetos Maven, mas o projeto também pode ser importado em Eclipse, NetBeans ou VS Code com suporte a Java e Maven.

Ferramenta ou componente	Versão ou requisito
Java Development Kit	JDK 21 ou superior.
Maven	Versão compatível com Java 21 e acesso ao Maven Central.
IntelliJ IDEA	Versão atual com suporte a Maven; Eclipse, NetBeans e VS Code também são alternativas.
Oracle Database FIAP	Serviço ORCL em <code>oracle.fiap.com.br:1521</code> .
Driver JDBC	<code>com.oracle.database.jdbc:ojdbc8:21.1.0.0</code> .
JUnit	JUnit Jupiter 5.10.2, executado pelo Maven Surefire.

8 8. Procedimentos para importar e executar

8.1 8.1 Importação no IntelliJ IDEA

Abra a pasta raiz do projeto, que contém `pom.xml`. Não abra somente a pasta `src`. No painel Maven, selecione **Reload All Maven Projects**. O IntelliJ deverá reconhecer automaticamente `src/main/java` como código de produção e `src/test/java` como código de teste.

Confirme se a dependência `ojdbc8` foi resolvida no classpath. Se o projeto for aberto em uma IDE diferente, importe-o como um projeto Maven existente a partir do arquivo `pom.xml`.

8.2 8.2 Configuração Oracle FIAP

Antes da execução do menu ou do teste manual, confirme o acesso à instância Oracle disponibilizada pela FIAP. O projeto não utiliza um Oracle local com XEPDB1; ele utiliza o host remoto e o SID ORCL definidos em `ConnectionFactory.java`:

```
URL: jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL
Usuário: rm570492
Senha: 030307
Driver: oracle.jdbc.OracleDriver
Serviço/SID: ORCL
```

Na primeira conexão, a aplicação tenta criar as tabelas ausentes no schema do usuário `rm570492`. O usuário precisa possuir permissões para criar e manipular as tabelas. A senha é documentada porque está definida diretamente no código-fonte conforme o requisito da entrega; em um ambiente real, ela deveria ser substituída por uma variável de ambiente ou um gerenciador de segredos.

8.3 8.3 Compilação

Na raiz do projeto, execute:

```
mvn clean compile
```

8.4 8.4 Execução do menu interativo

O menu executável está em:

```
src/main/java/br/com/EcoPulse/interfaces/Exibition.java
```

Sua classe principal é `br.com.EcoPulse.interfaces.Exibition`. Para iniciar com Maven:

```
mvn compile exec:java \
-Dexec.mainClass=br.com.EcoPulse.interfaces.Exibition
```

Também é possível abrir `Exibition.java` na IDE e executar o função `main`. O menu principal oferece gerenciamento de usuários, avatares, missões e a opção de saída.

8.5 8.5 Execução dos testes automatizados

Os testes automatizados estão em `src/test/java/br/com/EcoPulse/test`. Eles verificam regras de negócio, encapsulamento, estados, evolução, validações e lançamento de exceções. Não precisam de Oracle porque não executam operações JDBC.

Execute a suíte completa com:

```
mvn clean test
```

Para executar uma classe individual:

```
mvn -Dtest=UserTest test
mvn -Dtest=AvatarTest test
mvn -Dtest=MissionTest test
```

Os relatórios são gerados em `target/surefire-reports/`.

8.6 8.6 Execução do teste manual de CRUD

O teste manual exigido na entrega está em:

```
src/test/java/br/com/EcoPulse/test/UserCrudTest.java
```

Ele possui `main` e simula criação, leitura, atualização e exclusão de um usuário. Como está em `src/test/java`, execute com o classpath de testes:

```
mvn test-compile exec:java \
-Dexec.mainClass=br.com.EcoPulse.test.UserCrudTest \
-Dexec.classpathScope=test
```

Esse teste exige Oracle ativo. A saída esperada confirma CREATE, READ, UPDATE, DELETE e a mensagem de sucesso do CRUD.

9 9. Limites e observações

A aplicação é uma solução de console para fins acadêmicos e de demonstração de arquitetura Java. Ela não substitui um sistema de produção com autenticação, controle de acesso, gestão segura de segredos, migrações versionadas, observabilidade ou API externa.

As credenciais Oracle estão inseridas em código porque esse é um requisito da entrega. Em um ambiente real, elas deveriam ser obtidas por variáveis de ambiente ou um gerenciador de segredos. A limpeza automática dos registros ocorre somente quando a opção 0 é escolhida no menu principal; ela não deve ser acionada acidentalmente em um banco com dados que precisem ser preservados.

10 10. Referências

- [1] Maven Project, documentação oficial de build e gerenciamento de dependências: <https://maven.apache.org/> “Apache Maven Project”
- [2] Oracle, documentação do JDBC: <https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdbc/> “Oracle Database JDBC Developer’s Guide”
- [3] JUnit, documentação oficial do JUnit 5: <https://junit.org/junit5/docs/current/user-guide/> “JUnit 5 User Guide”